

PantryProof - Reflective Report

Domain understanding

One thing I learned while building PantryProof is that an ingredient being "replaceable" does not automatically mean a recipe can still be made. At first, I treated replaceable ingredients as less serious than essential ingredients. I assumed that if one was missing, the recipe could still be shown as possible with adjustments. Once I wrote the actual feasibility logic, I realised that was not always true. A replacement only helps if the cook actually has a suitable substitute in the pantry. This changed how I designed `RecipeEvaluation.feasibility`. The system now checks both the ingredient role and whether a real substitute is available. I added `test_feasibility_prefersBlocked_whenBothUnresolvedAndAdjustableIngredientsExist` to make sure a genuinely blocked ingredient is not hidden by another ingredient that only needs an adjustment. I made a similar decision with measurement units. The MVP does not guess conversions between units such as grams and tablespoons. If the quantities cannot be compared safely, `EvaluateRecipeFeasibilityUseCase` reports `quantityUnverified` and shows the pantry amount. It is less convenient, but I would rather show uncertainty than tell a cook they have enough when the app has not actually proved it.

Architecture decisions

In Assessment 1, I imagined the result working more like a conversation. The user could ask "Can I make this recipe?" and get an answer. After building the MVP, I changed my mind. A feasibility result should not depend on how an assistant happens to phrase an answer. The same pantry and the same recipe should give the same result every time. That is why the main decision is now handled by fixed business rules in `EvaluateRecipeFeasibilityUseCase` and `RecipeEvaluation.feasibility`. I separated the main operations because they protect different parts of the workflow. `EvaluateRecipeFeasibilityUseCase` protects the core rule: the app must not say a recipe is possible when an important ingredient is missing, insufficient, or cannot be verified. `UpdatePantryItemUseCase` protects the pantry state by letting the cook correct an existing item instead of creating conflicting entries. I also kept shopping-list operations separate so the next action stays clear when something needed for a recipe is missing. The ViewModels are deliberately thin. They handle presentation state, while validation and business rules stay in Use Cases and domain models, where I can test them without SwiftUI. I also ran into a real architecture problem during testing. `PantryViewModelTests` crashed with a `libmalloc double-free error` because the Xcode 26 setting `SWIFT_DEFAULT_ACTOR_ISOLATION = MainActor` was isolating classes that did not need it. Removing that default and marking only the ViewModels `@MainActor` fixed the crash and made the responsibility of each layer clearer.

Human-system design

A realistic mistake in PantryProof is that a cook may try to add an ingredient that is already in the pantry because they forgot it was recorded earlier. My first version detected the duplicate and told the user to update the existing item instead. The message was correct, but there was a simple problem: there was no edit screen. So the app was telling the user what to do without giving them a way to do it. I fixed this by adding `UpdatePantryItemUseCase` and connecting the duplicate error to an edit sheet. Now the error has a real recovery path. I found the same kind of problem later when a recipe with no ingredients told the user to choose another recipe, but there was no direct way back to the recipe list. I added that route too. These changes made me think differently about error handling. A useful error is not just a correct sentence. The user should understand what happened and be able to do something about it immediately. `RecipeDetailView` follows the same idea by getting its guidance from the same `RecipeEvaluation` as the ingredient list, so the screen cannot give two conflicting answers about the same recipe.

What I would do next

If I had another two weeks, the one architectural change I would make is persistent storage. `PantryRepository` and `ShoppingListRepository` are already abstracted, so I would replace the in-memory implementations with `SwiftData`-backed repositories without changing the Use Cases. This matters because the current MVP can prove the workflow, but a real cook would expect their pantry and shopping list to still be there after closing the app. Losing that information would make the product hard to trust and annoying to use. I would keep the repository interfaces the same and make persistence a data-layer change, then add tests for saving, loading, and updating records. That would also be a good test of the architecture itself. If I can add persistence without rewriting the business rules or the UI flow, then the separation between the layers is doing its job.